

AI-Powered Behavioral Anomaly Detection for Cybersecurity

Honeywell Hackathon - Q4 - Technical Report

Summary. Signature-based security misses novel and slow attacks. We built a behavioral anomaly-detection system that learns a per-entity model of "normal" - for users, service accounts, and industrial edge devices - from synthetic access logs we generate with eight injected behavior patterns. An incremental feature layer feeds a two-stage, label-blind detector (Isolation Forest fused with a GRU sequence autoencoder); a hybrid classifier names the likely attack type and escalates unrecognized behavior as novel; and an explainability layer gives the analyst a one-sentence reason with a counterfactual for every alert. We optimize for the metric that matters in a real Security Operations Center (SOC) - true positives inside the small top-percentage of events an analyst can actually review - and we handle cold-start entities and legitimate concept drift so the system does not raise false alarms on benign change. Every feature is computable incrementally, so the design runs on a live stream.

1. Problem framing

Stripped of security vocabulary, the task is: learn what "normal" looks like for each entity, then flag and explain behavior that deviates - on streaming, time-ordered, heavily imbalanced data - and say which kind of attack the deviation resembles. We anchor the narrative in OT and industrial-edge security (SCADA/HMI consoles, PLC gateways, historians, service accounts, edge devices) to match Honeywell's domain.

The security vocabulary is really just the label set; the underlying ML problem is per-entity baselining + sequence anomaly detection + attack attribution + explainability. Our design goal follows from that: **a system a real SOC analyst would actually trust and use - accurate on rare events, low false-positive at a realistic alert budget, and explainable per alert.**

The graded challenges and how we address each:

Challenge	Our approach
Sequential / behavioral data	GRU autoencoder over each entity's event sequence, plus per-entity rolling features
Extreme class imbalance	Anomaly-detection framing; report PR-AUC and recall at budget, never accuracy
Concept drift	Adaptive per-entity baselines; explicit benign insider-drift edge case
Explainability	Per-alert attribution (SHAP + reconstruction error) to a plain-language reason and counterfactual
Cold-start	Neutralize personal-history features for new entities; score on universal signals

Coverage of the seven judging criteria. (1) Detection accuracy on imbalanced labels - PR-AUC and recall at budget (§6); (2) attack-type classification - hybrid classifier (§4); (3) false positives at a realistic analyst budget - alert-budget metric and de-duplication (§6, §8); (4) explainability / analyst usability - per-alert reason +

counterfactual and the dashboard (§5); (5) cold-start and concept drift - handled explicitly, including the insider-drift trap (§7); (6) system design and scalability - fully incremental, stream-ready features (§8); (7) report clarity - this document. We deliberately cover all seven rather than over-investing in two, since the three most-skipped by competitors - false-positives-at-budget, explainability, and streaming design - are exactly where a submission differentiates.

2. Synthetic data - methodology and assumptions

Fleet. 100 entities (60 users, 25 service accounts, 15 edge devices) over 30 days, producing about 47,000 time-ordered events.

Schema (as mandated): `entity_id`, `entity_type`, `timestamp`, `source_ip`, `geo_location`, `resource_accessed`, `auth_method`, `session_duration`, `command_sequence`, `device_fingerprint`, `label`.

Per-entity baseline. Each entity is generated from a fixed profile: habitual login hours, a home city, a small home-IP pool, a usual resource set, a stable device fingerprint, a session-duration distribution, and a daily event rate.

Documented behavioral assumptions:

1. *Failed-auth encoding.* The schema has no success/failure column, so a failed authentication is encoded as `resource_accessed = auth/login` with `session_duration = 0` (a login that never opened a session). This makes "failed-auth count in a window" a simple incremental counter.
2. *Sticky per-day geography.* A real entity is in one city per day and does not teleport between consecutive logins. Location is fixed once per day, so benign geo-velocity stays plausible and only the injected impossible-travel pattern produces superhuman speeds.
3. *Deliberate benign noise.* Off-hours logins, legitimate travel, first-time resource access, and mistyped-password failures overlap the attack classes so the metrics are honest rather than trivially optimistic.
4. *command_sequence is populated only for privileged sessions*, matching the schema note and making it the key signal for lateral movement and exfiltration.

Injected behaviors (about 1.4 percent of events). Labels are written to a separate file, so the event log used for inference is label-blind by construction (an assertion enforces this).

Behavior	Label	Detection handle
normal	0	habitual profile plus noise
brute_force	1	burst of failed logins from one IP on one account
impossible_travel	1	geo-velocity between logins over 900 km/h
credential_stuffing	1	few IPs to many accounts (IP fan-in)

lateral_movement	1	resource-novelty plus recon commands
device_spoofing	1	device fingerprint mismatch
low_and_slow	1	off-hours sensitive access with egress commands
insider_drift	0	benign slow growth: in-hours, home geo, internal, no egress

The insider-drift trap. Insider drift and low-and-slow are both gradual expansions, which is why naive detectors confuse them. We bake four discriminators into the data and verified them on the generated set:

Signal	low_and_slow (attack)	insider_drift (benign)
off-hours	100%	0%
egress command	100%	0%
sensitive resource	100%	45%

The 45 percent is intentional: sensitive access alone would false-flag the benign drift; the off-hours and egress signals are what separate them.

Reproducibility. A single seed drives NumPy and Faker; the generated data is byte-for-byte identical across runs.

3. Feature layer (incremental, streaming-ready)

A single causal pass maintains a small per-entity and per-IP running state and emits nine features per event. Each uses only past events, so the same code runs on a live stream.

Feature	Catches	Incremental state
geo_velocity_kmh	impossible travel	last (city, time)
failed_auth_in_window	brute force / stuffing	per-IP 5-min window
resource_novelty	lateral movement	per-entity resource set
device_mismatch	device spoofing	per-entity device set
off_hours	odd-hour activity	per-entity hour histogram
duration_zscore	unusual sessions	online mean / variance
cmd_novelty	recon / egress commands	per-entity token set
ip_fanin	credential stuffing	per-IP 10-min window

cold_start	new-entity routing	per-entity event counter
------------	--------------------	--------------------------

Because these features are entity-relative, one global detector behaves like a per-entity model without training 100 separate models.

4. Models - what, where, and why

Two-stage detection (both unsupervised and label-blind):

1. *Isolation Forest (baseline profiler)*. Fast, robust, isolates outliers in the nine-feature space; strong on single-event spikes such as brute force. PR-AUC about 0.42 on its own.
2. *GRU sequence autoencoder (sequence-aware detector)*. For each event it reconstructs the window of the entity's last 16 events; high reconstruction error means the behavior is surprising given recent history. A GRU is chosen over LSTM or Transformer because it captures order, trains in seconds, and needs little data. It recovers the order-dependent attacks the forest dilutes.

Fusion. Each detector's score is normalized against the training-period distribution (streaming-honest) and combined by elementwise maximum, so an attack detected by either model stays flagged. Averaging was rejected because it diluted the brute-force spike (recall 1.00 to 0.18).

Attack classifier (three-stage hybrid).

1. *Rules* for well-defined signatures (geo-velocity over 900 to impossible travel; device mismatch to spoofing; recon commands to lateral movement; egress commands to exfiltration; IP fan-in to stuffing; failed-auth burst to brute force). Rules only name an event the label-blind detector already flagged.
2. *RandomForest*, trained on labeled signatures, for the remainder. It is robust and supports exact TreeSHAP attributions.
3. *Novel / unknown*. An anomalous alert matching no rule that the model can only call benign or low-confidence is escalated rather than guessed. Verified by holding an attack family out of training: 100 percent of its anomalous events routed to novel.

5. Explainability

For each alert the top-three contributing features are ranked from three sources: the rule that fired, TreeSHAP on the classifier, and per-feature reconstruction error from the GRU. These are rendered into a deterministic, templated sentence with a counterfactual. No language model is used, so the output cannot hallucinate.

Flagged (risk 1.00, lateral_movement): unfamiliar commands (login, whoami, net_view, enum_shares) - 100% never seen for this entity; session length +1.1 std from this entity's norm. Counterfactual: would not alert if only familiar commands were used.

6. Metrics (imbalance-robust, never accuracy)

Evaluated on a temporal test split (train on the first 70 percent of the timeline, score the last 30 percent), about 1.3 percent anomalies.

Metric	Value
PR-AUC (unified detector)	0.42 (random 0.011)
ROC-AUC	0.97
Recall at top-2% budget	0.67
Recall at top-5% budget	0.87
Attack-type naming (in budget)	1.00
Calibration Brier (rank to isotonic)	0.27 to 0.007

Per-attack recall at the 2 percent budget (coverage across categories, not just the frequent ones):

Attack type	Recall
device_spoofing	1.00
impossible_travel	1.00
brute_force	0.82
credential_stuffing	0.64
lateral_movement	0.50
low_and_slow	0.29

Why not accuracy: predicting "normal" for every event scores about 98.7 percent accuracy while catching zero attacks. We report accuracy only to show why it is misleading.

Figures (in figures/): `pr_curve_final.png`, `alert_budget_final.png`, `confusion_matrix.png`, `calibration.png`. All are generated by `models/metrics.py` from the real test-set predictions.

7. Cold-start and concept drift

Cold-start. For an entity with fewer than five events, the personal-history features are neutralized and only universal signals (geo-velocity, failed-auth, IP fan-in) are used; a cold-start flag marks the score as low-confidence. New entities are not blanket-flagged.

Concept drift. Per-entity baselines are running and decaying, so they absorb legitimate change. The insider-drift case proves it: a legitimate user expanding their footprint stays benign (0 of 172 flagged), because it is in-hours, home geo, internal resources, no egress - while the look-alike low-and-slow exfiltration is still caught.

8. Streaming and scalability design

```
event stream
-> feature service (9 incremental per-entity features)
-> detector: Isolation Forest + GRU autoencoder (label-blind)
-> classifier: rules + RandomForest + novel/unknown
-> explainer: SHAP + reconstruction error -> reason + counterfactual
-> ranked, explained alert queue (+ analyst feedback)
```

Every feature is a rolling count, a running position, a decaying profile, or a sliding-window statistic - all incrementally maintainable. Per-entity state is small and shardable by entity, so it can be cached in a store such as Redis. The detector scores one event at a time; the classifier and explainer are per-event. We demo in batch; the architecture streams. Operational maturity features: isotonic-calibrated risk, alert de-duplication (a 25-event burst becomes one incident), and an analyst feedback loop.

9a. Known limitations

Two caveats scope the results honestly; each already has a mitigation in the design.

1. *Synthetic, single-seed data.* All results are on data we generated from one seed, with no cross-dataset or real-world validation. Real logs are noisier and more adversarial and their signatures (especially command sequences) are messier, so the absolute numbers would move - we would expect lower rule precision and would retrain with harder negatives. Read the numbers as evidence the approach works, not as production benchmarks.
2. *Batch demo.* Every feature is incremental and the architecture streams, but the demo runs in batch against no live broker, so streaming latency and the periodic refitting the isotonic calibration would need under drift are designed-for, not yet measured.

9b. Future work

Peer-group cold-start scoring (cluster similar entities), online autoencoder updates so baselines adapt continuously, graph features for lateral movement across the entity-resource graph, adversarial/noisier attack models for harder evaluation, and closing the feedback loop with active learning from analyst confirm/dismiss decisions.

10. How to reproduce

```
python -m pip install -r requirements.txt
python run_pipeline.py
python -m streamlit run dashboard/app.py
```

A fixed seed makes the data and metrics reproducible. Outputs land in `data/`, `models/`, and `figures/`, and are viewable in the dashboard.